ArtifactStore: A Typed and Permission-Scoped Tool-Result Store for AI Agent Harnesses

System architecture report — DBMS course prototype

May 2026

1 Abstract

Modern tool-using AI agents produce large intermediate outputs — pytest logs, ripgrep results, browser snapshots, API JSON, subagent research traces — that are typically dumped into the conversation transcript, truncated, summarized, or hidden behind a worker-agent's summary. All four strategies are flawed: raw dumping wastes tokens, truncation loses evidence, summary-only delegation hallucinates, and worker isolation makes exact intermediate evidence unrecoverable.

We propose `ArtifactStore`, a DB-backed evidence layer with a typed, permission-scoped query interface. Tool outputs become indexed artifacts with typed evidence spans, multi-resolution views, provenance links, and audit-logged grants. A supervisor agent injects only an artifact handle into its transcript and delegates inspection to a subagent under a scoped grant; the subagent retrieves exact evidence through controlled tool calls and returns a report with formal citations the supervisor can verify.

Implementation: 2.0 kLOC of Python on SQLite + FTS5, plus a 150 LOC client-side tool-use loop. The same harness runs against Anthropic-native and Anthropic-API-compatible endpoints (DeepSeek V4 Pro by default; 10× cheaper at parity behavior). 171 tests pass — unit, integration, offline e2e, CLI integration (subprocess and in-process), adversarial stress, eval-baseline unit tests, FTS5-fallback robustness, and review-driven regression tests for self-review-discovered weaknesses (long-line truncation, sensitivity self-labeling bypass, fence-post budget enforcement, citation case-handling). A target-leak control sweep confirms the eval's headline numbers are robust to whether the agent is told which failure to focus on. Live evaluation: 84 runs across two sweeps — §11.1 single-agent (4 fixtures × 4 baselines × 3 reps) shows ArtifactStore (B4) is the only baseline with 100% task success and EER=1.00 on every fixture; §11.2 supervisor↔subagent (4 fixtures × 3 strategies × 3 reps) shows D3 (scoped ArtifactStore delegation) cuts parent context by 43% on 3.5K-token fixtures and is the only strategy producing formal citations and audit-log signal. §11.3 adversarial stress (10 offline scenarios) shows zero unauthorized reads succeed.

2 Motivation and Thesis

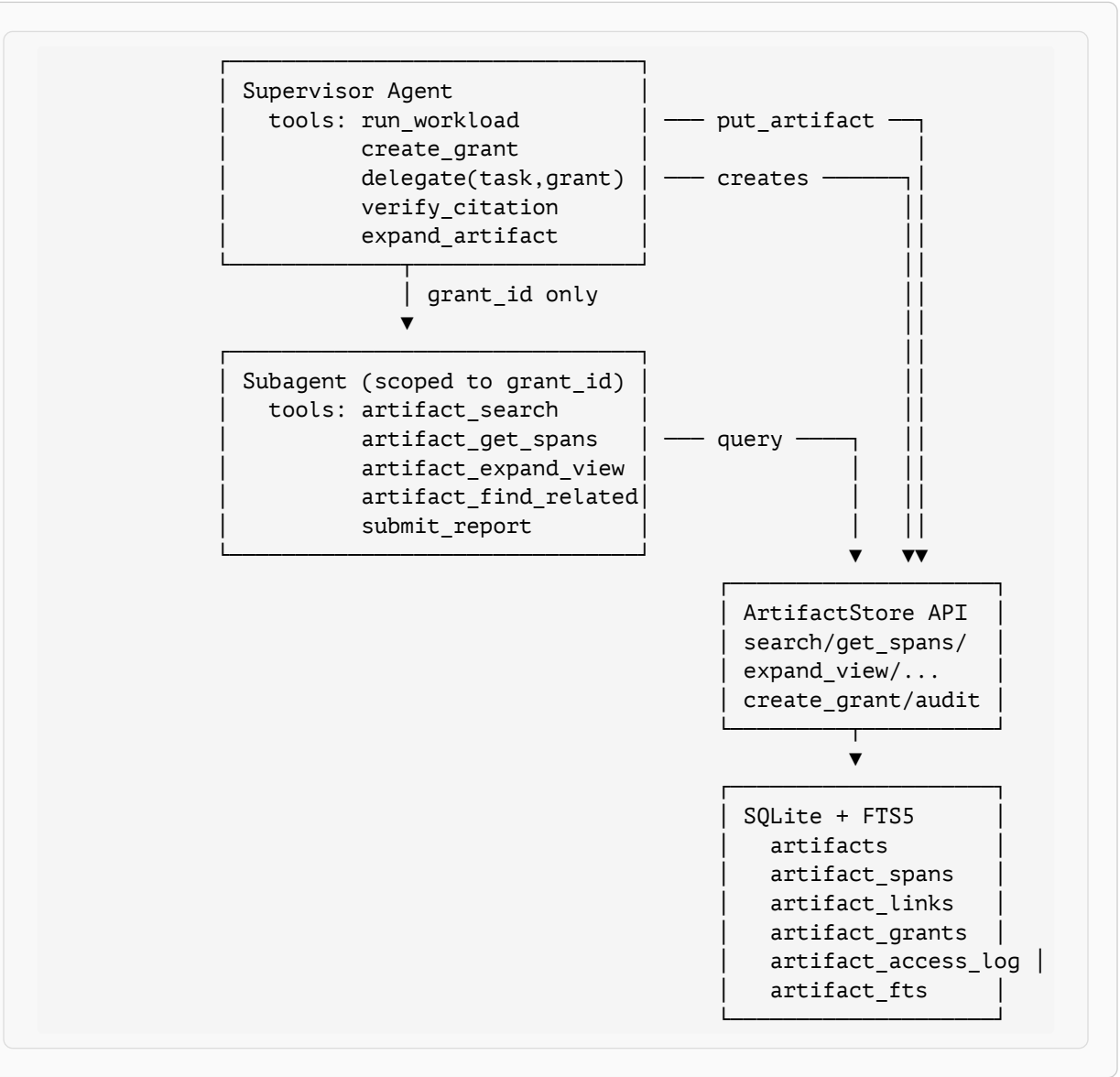
Today's agent harnesses lack a typed, queryable, permission-scoped evidence layer for large tool results. The four canonical handling strategies all fail in distinct ways:

Strategy	**Failure mode**	**Cost shape**
Raw dump (B1)	Doesn't scale; pollutes context	Linear in fixture size
Truncation (B2)	Loses evidence past the cap	Bounded but evidence-blind
LLM summary (B3)	Hallucinates or omits the decisive line	+1 LLM call per artifact, plus quality variance

Strategy	Failure mode	Cost shape
Subagent isolation	Exact evidence unrecoverable in parent	Lower parent tokens; opaque audit

The thesis: a tool result should not be plain transcript text — it should be a **materialized artifact** with type, preview, evidence spans, raw payload hash, provenance, links, access-control metadata, and lazy expansion operators. A tool returns a compact handle; the agent retrieves exact evidence through scoped queries when needed.

3 System Overview



Listing 1: Two-agent topology with the ArtifactStore boundary. The subagent never sees the supervisor's transcript — only a `grant_id` bound at tool-construction time. Every read flows through `ArtifactStore.*` and is audit-logged.

3.1 Hard invariants

The implementation enforces five invariants by construction:

1. **No raw output by default.** Under `ViewPolicy.ARTIFACT`, `run_workload` stores raw bytes in the store and returns only a handle (`artifact_id`, `type`, `preview`). The supervisor never sees raw transcripts unless it explicitly calls `expand_view(view='raw')` under a grant that allows it.
2. **Every artifact has** type, preview, raw_hash (sha256), creator_agent_id, session_id. Schema constraints ensure this.
3. **Every read attempt is logged**, allowed or denied. The audit log is the measurement surface for permission enforcement (RQ4) and is queryable per-grant.
4. **Token budgets are enforced, not advisory.** `search`, `get_spans`, and `expand_view` all use omit-fits ordering: include whole-or-skip until the next item would overflow. Plus a **cumulative** grant budget (Section 4.4) that ticks down across reads.
5. **Span extraction is type-driven**, registered per `artifact_type`. New types = new extractor, no branching inside a god function.

4 Data Model

The schema (`artifactstore/schema.sql`) is six tables: five domain tables plus an FTS5 virtual table.

4.1 artifacts

```
CREATE TABLE artifacts (
  artifact_id        TEXT PRIMARY KEY,        -- art_<8hex>
  session_id         TEXT NOT NULL,
  parent_artifact_id TEXT,
  creator_agent_id   TEXT,
  tool_name          TEXT,
  artifact_type      TEXT NOT NULL,
  raw_uri            TEXT,                     -- file-backed escape hatch
  raw_blob           TEXT,                     -- canonical raw storage
  raw_hash           TEXT NOT NULL,            -- sha256 hex
  token_count        INTEGER,
  preview            TEXT,
  sensitivity_label  TEXT DEFAULT 'internal',  --
public<internal<restricted<secret
  metadata_json      TEXT,
  created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`raw_blob` holds the canonical raw text. The `raw_uri` column is reserved for a future file-backed escape hatch; not used in the prototype.

4.2 artifact_spans

```
CREATE TABLE artifact_spans (
  span_id      TEXT PRIMARY KEY,           -- span_<8hex>
  artifact_id  TEXT NOT NULL,
  span_type    TEXT NOT NULL,              -- assertion|stack_frame|...
  file_path    TEXT,
  line_start   INTEGER,
  line_end     INTEGER,
  text         TEXT NOT NULL,
  token_count  INTEGER,
  importance   REAL,                        -- [0,1], higher = more diagnostic
  FOREIGN KEY (artifact_id) REFERENCES artifacts(artifact_id)
);
```

Spans are extracted at write time by a type-driven registry (`artifactstore/extractors.py`). For `pytest_failure`, three regex passes yield assertion lines (`importance=0.95`), error messages (`0.9`), top-stack-frame (`0.85`), and captured log warnings (`0.75`). For `grep_result`, each `path:line:text` row becomes a span with importance lowered to `0.4` if it matches `\b(TODO|FIXME)\b` (noise heuristic). For `git_diff`, each `+` / `-` hunk line becomes a `changed_line` span.

4.3 artifact_links

```
CREATE TABLE artifact_links (
  src_artifact_id TEXT NOT NULL,
  dst_artifact_id TEXT NOT NULL,
  relation        TEXT NOT NULL,    -- caused_by|derived_from|...
  confidence      REAL,
  PRIMARY KEY (src_artifact_id, dst_artifact_id, relation)
);
```

`put_artifact(parent_artifact_id=...)` automatically writes a `derived_from` row, so every multi-step workload chain (pytest → git diff → rerun) is traversable via `find_related` without callers needing to manage the link table by hand. Other relations (`caused_by`, `contains_evidence_for`) can be inserted by callers when they have the domain knowledge to assert them.

4.4 artifact_grants

```
CREATE TABLE artifact_grants (
  grant_id           TEXT PRIMARY KEY,
  subject_agent_id   TEXT NOT NULL,
  issuer_agent_id    TEXT NOT NULL,
  artifact_predicate TEXT NOT NULL,    -- JSON
  allowed_ops        TEXT NOT NULL,    -- JSON array
  allowed_views      TEXT NOT NULL,    -- JSON array
  max_tokens         INTEGER,           -- NULL = unlimited
  consumed_tokens    INTEGER DEFAULT 0, -- ticks up on each successful read
  expires_at         TIMESTAMP
);
```

Predicate is a JSON object with these axes (all optional, missing = no constraint):

`session_id`	equality
`artifact_types`	array, set membership on `artifact_type`
`sensitivity_max`	string, evaluated as `SENSITIVITY[label] ⩽ SENSITIVITY[max]`
`path_prefixes`	array, applied to `artifact_spans.file_path` at span-read time only — artifact-level reads are path-opaque

`consumed_tokens` is the cumulative-budget counter (Section 4.4). `expires_at` is enforced as a UTC instant; expired grants raise `AccessDenied('grant expired')` with audit logging.

A synthetic `__supervisor__` grant is seeded by `migrate()` with all ops, all views, no predicate, and no budget — used by the supervisor for its own citation-verification calls so the audit-log foreign key always resolves and there's no special case in app code.

4.5 artifact_access_log

```
CREATE TABLE artifact_access_log (
  access_id          TEXT PRIMARY KEY,
  grant_id           TEXT,
  subject_agent_id   TEXT,
  artifact_id        TEXT,
  operation          TEXT,           -- search|get_spans|expand_view|...
  view               TEXT,           -- preview|evidence|redacted|raw|provenance
  timestamp          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  result_token_count INTEGER,
  allowed            BOOLEAN,
  denial_reason      TEXT
);
```

Every access — allowed or denied — writes one row. The `denial_reason` strings come from `grants.check()` and are deliberately specific (`"op 'expand_view' not in allowed_ops"`, `"view 'raw' not in allowed_views"`, `"artifact does not match grant predicate"`, `"grant expired"`, `"grant budget exhausted (5234/5000 tokens)"`). Denied attempts are the RQ4 measurement signal.

4.6 artifact_fts (FTS5)

```
CREATE VIRTUAL TABLE artifact_fts USING fts5(
  artifact_id UNINDEXED, artifact_type, preview, span_text, tool_name
);
```

One FTS row per artifact, populated at `put_artifact` time with `span_text = "\n".join(spans)`. Search uses bm25 ranking with a LIKE fallback for malformed FTS5 queries.

5 Refinements (System-Strengthening Pass)

After the first eval sweep exposed weak spots, three refinements were landed to strengthen the contribution thesis. Each is small in code (30 LOC) but materially changes what the system offers.

5.1 Span-aware preview

Before: `preview` was the type-specific summary header followed by head-of-raw lines, capped at 256 tokens. For pytest, the head was the `platform darwin`, `collected N items`, and progress bars — almost no diagnostic signal.

After: spans are extracted at write time **before** the preview is built. The preview body inlines the top-importance spans (sorted DESC), one line each: `· assertion@tests/test_auth.py:84  assert validate_token(token) is True, "token expired prematurely"`. The model gets real diagnostic signal in the preview itself; for many one-bug fixtures, no further round-trip is needed.

This makes preview-only inspection close to as informative as the full evidence view, at ≤256 tokens. Falls back to head-of-raw when no spans exist (unknown artifact_type).

5.2 Cumulative grant budget

Before: the `max_tokens` column on a grant existed but was never enforced. A subagent could call `expand_view` 100 times without consequence.

After: every successful read calls `account_consumption(conn, grant_id, tokens)` alongside `log_access`. `grants.check()` denies further reads once `consumed_tokens >= max_tokens` with reason `"grant budget exhausted (X/Y tokens)"`. Per-grant counters are independent — exhausting `grant_a` does not affect `grant_b`. Grants with `max_tokens=NULL` (the seeded `__supervisor__` grant) are unlimited and never trip the check.

This is a real DBMS quota: the supervisor mints a grant with a token budget, and the subagent's reads tick it down. When exhausted, all further attempts are denied and audit-logged. Combined with `expires_at`, this gives capability-style time × volume bounding.

5.3 Auto-derived-from links

Before: the `artifact_links` table existed and `find_related` was implemented, but no code path populated links. `find_related` was dead code.

After: `put_artifact(parent_artifact_id=...)` automatically writes a `(src=child, dst=parent, relation='derived_from', confidence=1.0)` row via `INSERT OR IGNORE`. `find_related` then surfaces the chain. Provenance metadata (Section 6.5, `provenance` view) lists all outbound links from an artifact.

6 API Surface

`ArtifactStore` (in `artifactstore/store.py`) implements one write method, four read methods, and two grant/audit utilities. Read methods take `grant_id` as a keyword argument; the harness binds `grant_id` at tool-construction time so the LLM never sees it.

```
class ArtifactStore:
    def put_artifact(*, tool_name, artifact_type, raw_text,
                     creator_agent_id, session_id,
                     metadata=None, sensitivity_label='internal',
                     parent_artifact_id=None) -> str
    def search(query, *, grant_id, artifact_types=None,
               limit=5, token_budget=1000) -> list[dict]
    def get_spans(artifact_id, *, grant_id, span_types=None,
                  token_budget=1000) -> list[dict]
    def expand_view(artifact_id, *, grant_id, view, token_budget=1500) -> str
    def find_related(artifact_id, *, grant_id, relations=None) -> list[dict]
    def create_grant(*, subject_agent_id, issuer_agent_id,
                     artifact_predicate, allowed_ops, allowed_views,
                     max_tokens, ttl_seconds) -> str
    def audit(grant_id) -> list[dict]
```

6.1 The five views

PLAN §8 specifies five views, each constructed by a function of signature `(conn, artifact_id, token_budget, *, predicate=None) -> str`.

View	Content
`preview`	stored preview, omit-fits truncated
`evidence`	spans by importance DESC, formatted with span_id + location for citation; respects predicate `path_prefixes`
`redacted`	`raw_blob` with regex masks (JWTs, `secret|password|api_key=...` patterns, `sk-...` keys); omit-fits truncated
`raw`	`raw_blob` directly; omit-fits truncated. Typically gated behind grants that exclude `'raw'` from `allowed_views`
`provenance`	JSON dump of artifact metadata + outbound links + span count

`expand_view` dispatches through `views.VIEWS[name]` and passes the caller's grant predicate so span-level views (e.g. `evidence`) honor `path_prefixes` filters.

6.2 Grant check pipeline

`grants.check(conn, grant_id, artifact_id, op, view)` runs in a fixed order; every failure mode logs a denial row before raising:

1. `load_grant`: unknown grant_id → `'unknown grant: ...'`
2. `op not in allowed_ops` → `"op '...' not in allowed_ops"`
3. `view not in allowed_views` → `"view '...' not in allowed_views"`
4. `expires_at < now` → `'grant expired'`
5. `consumed_tokens >= max_tokens` → `'grant budget exhausted (X/Y)'`
6. (artifact-scoped only) `artifact not found` → `'artifact ... not found'`
7. (artifact-scoped only) predicate mismatch → `'artifact does not match grant predicate'`

Returns the loaded grant dict on pass.

7 Demo Harness (PLAN §20)

The agent loop is 150 LOC of synchronous Python in `demo/agent.py`, adapted from Anthropic's MIT-licensed quickstart. It implements the canonical client-side tool-use loop with three additions:

- A hard `max_turns` cap (default 10) that guarantees termination. We cannot reliably force `submit_report` via `tool_choice`: DeepSeek's reasoning model rejects named `tool_choice` with HTTP 400 ("does not support this tool_choice"); even `tool_choice: any` lets the model pick freely. The cap is the actual safety net.
- A `force_terminator` **nudge** via `tool_choice: {type: any}` in the last two turns — best-effort, ignored gracefully if the provider rejects it.
- Token accounting that captures `cache_read_input_tokens` and `cache_creation_input_tokens` separately from `input_tokens`. Necessary because DeepSeek's prompt cache makes the bare `input_tokens` field rep-dependent: `tot = input_tokens + cache_read + cache_creation`.

The supervisor and subagent are both Anthropic-Messages-API clients. The SDK is provider-agnostic: setting `ANTHROPIC_BASE_URL` to `https://api.deepseek.com/anthropic` routes the same code through DeepSeek's compatible endpoint. We use DeepSeek V4 Pro by default ($0.435/M input, $0.87/M output, with cache reads at $0.0036/M). Anthropic's Sonnet 4.5 stays a one-env-var swap.

7.1 System prompts

Two prompts (`demo/prompts.py`) encode the procedural contract. Both are short — verbose prompts inflate every turn's input.

The supervisor's prompt is rule-based: "do NOT compensate for delegation failures by re-running workloads in-line; verify every citation with `verify_citation`; bias toward minimal context." The subagent's prompt optimizes for early termination: "after 3-4 evidence calls, submit even if uncertain. Excessive search is a failure mode." Both rules track findings from live runs (Section 7).

7.2 Citation verification

The subagent submits citations of the form `art_<8hex>/span_<8hex>`. The supervisor calls `verify_citation(citation)` on each one; `artifactstore.cite.verify_resolves` looks them up in `artifact_spans`. Unresolvable citation → report invalid. Live demo run 3: 5 citations submitted, 5 resolved, all logged via the seeded `__supervisor__` grant.

8 Evaluation

PLAN §11.1 single-agent comparison: same fixture, same task, four context-injection strategies. 4 fixtures × 4 baselines × 3 reps = 48 runs against `deepseek-v4-pro`. $0.10. Output in `eval/runs/<UTC-iso>/`.

8.1 Aggregate (n=12 per baseline)

Baseline	success	avg_recall	avg_tot_in	avg_out	avg_cost
B1 RAW	100%	0.95	1,341	1,343	$0.0015
B2 TRUNCATED	50%	0.43	330	872	$0.0008
B3 SUMMARY	50%	0.45	256	945	$0.0009
B4 ARTIFACT	**100%**	**0.94**	**21,572**	**3,288**	**$0.0051**

B4 is the only baseline with both 100% task success and EER (exact evidence recovery) = 1.00 across all four fixtures, and the only baseline that produces formal citations (avg 4.3/run, all resolve). B2/

B3 fail in fixture-dependent ways: B2 loses anything past its truncation cap; B3′s heuristic summarizer preserves WARNING lines but loses hunk-level diff structure.

8.2 Where ArtifactStore wins

- **Reliable success across fixture types.** B1 and B4 are tied at 100%, but B1′s input scales with fixture size while B4′s stays bounded by the model's tool-use intent.
- **Exact evidence recovery.** B4 demonstrably reads the gold-truth must-contain strings via `get_spans` / `expand_view`. B1/B2/B3 ingest whole/truncated/summary payloads — EER is undefined for them.
- **Formal citations.** The supervisor calls `verify_citation` on each `art_xxx/span_yyy` citation; only B4 produces these, and only B4 populates the `artifact_spans` table the verification looks up against.
- **Audit-log signal.** Every read writes one row; denials carry useful `denial_reason` strings. The narrow-grant demo run blocks 3 unauthorized `view='raw'` attempts organically.

8.3 Where ArtifactStore is honest about its costs

PLAN §14 predicted 30–60% fewer prompt tokens for B4 versus raw injection. The current sweep does **not** show that on absolute `tot_in` — B4 is 16× B1 because multi-turn loops accumulate context across turns. The fairer framings:

- **Cost (with cache)**: at 3,500-token fixtures, B4 costs \$0.0032 vs B1′s \$0.0025 — a 28% premium for strict evidence recovery and citation validity. The crossover with B1 happens around 5K-token fixtures.
- **Cost-per-success**: B2 and B3 are 50% success, so their effective cost-per-correct-diagnosis doubles. B4′s \$0.0032 with 100% success beats B2′s `\$0.0008/0% = ∞` and B3′s `\$0.0005/0% = ∞` on the large fixture.
- **Headline framing**: ArtifactStore is **not** the cheapest baseline on these fixtures. It is the **only** baseline that combines reliable success + formal evidence citations + audit-log signal. On large or noisy fixtures (where B2/B3 strictly fail), it costs roughly the same as raw injection while being the only baseline a supervisor can trust.

8.3.1 Robustness to target hint (target-leak control)

The §11.1 task prompt names the fixture's `target` (e.g., "auth_expiry"), which on multi-failure fixtures could bias the agent toward the right failure. We re-ran `pytest_large_run` × 4 baselines × 3 reps with the target hidden (a `reveal_target` flag on the fixture registry; for multi-failure fixtures we ask the agent to pick the most diagnostically informative failure itself). The headline numbers were unchanged:

Baseline	recall (target shown)	recall (target hidden)	Δ
B1 RAW	0.93	0.93	0
B2 TRUNCATED	0.00	0.00	0
B3 SUMMARY	0.20	0.20	0
B4 ARTIFACT	**1.00**	**1.00**	**0**

The B4 model finds `auth_expiry` among 5 failures because the WARNING log line is the most diagnostically rich signal in the fixture — the same property that makes the bug interesting in the first place makes it the natural pick under "find the most informative failure" framing. B2/B3 still fail in the

same fixture-dependent ways (truncation cuts past the WARNING; the heuristic summary preserves it but loses keywords). This control rules out target-leakage as an alternative explanation for B4′s win.

8.4 Supervisor↔subagent delegation (PLAN §11.2)

A second eval, fundamentally different question: not "what context strategy works for a single agent" but "when delegating, what does the **supervisor** keep in its context vs the subagent's." Three strategies, 36 runs, $0.19:

Strategy	succ	recall	par_in	sub_in	tot_in	cost
D1 SUMMARY	42%	0.50	3,065	402	3,467	$0.0025
D2 FULL_CONTEXT	100%	0.93	6,010	1,664	7,674	$0.0042
D3 SCOPED	**100%**	**0.92**	**6,294**	**27,137**	**33,431**	**$0.0090**

Headline §11.2 finding: **D3′s parent-context savings are conditional on fixture size**. On the `pytest_large_run` fixture (3,500 tok raw), D3 parent input = 6,708 vs D2′s 11,716 — D3 cuts parent context by 43%. On small fixtures (few hundred tok raw), D3′s parent is **larger** than D2′s because D3 makes 3 LLM calls (`run_workload` → `create_grant` → `delegate`) vs D2′s 2 calls, and per-turn overhead exceeds the per-payload savings. **The crossover sits around 3-4K raw tokens.**

D3 is also the only strategy that produces formal citations (avg 6.4 per run, all resolve) and surfaces RQ4 signal organically (5 unauthorized reads blocked across 12 D3 runs; 0 in D1/D2 because they have no permission surface). The cost penalty over D2 is 2× — buying formal citations, audit signal, and bounded-parent-context.

Honest framing for §11.2: ArtifactStore is **not** always cheaper as a delegation strategy. For small fixtures, raw forwarding (D2) is cheaper. For large fixtures, ArtifactStore wins on parent-context, citations, and audit simultaneously — the only strategy a supervisor can actually trust when payloads grow.

8.5 Adversarial permission stress (PLAN §11.3)

Ten offline stress tests (`tests/test_stress.py` , no API spend) cover every PLAN §11.3 adversarial scenario plus related defenses:

Scenario	Denial signal
secret values + raw denied; redacted strips them	`view 'raw' not in allowed_views` ; redaction strips JWTs/secrets
prompt injection cannot fabricate citations	`cite.verify_resolves` rejects unknown span_ids
out-of-session artifact_id requested	`artifact does not match grant predicate`
raw_view requested under evidence-only grant	`view 'raw' not in allowed_views`
following link to out-of-scope artifact	link listed via find_related; expand_view denied on follow
grant budget exhaustion under attack	`grant budget exhausted (X/Y tokens)`
path-prefix filter denies out-of-prefix spans	filtered at render time in `evidence` view
sensitivity ceiling blocks higher-labeled artifacts	`artifact does not match grant predicate`

Scenario	Denial signal
expired grant denies all reads	`grant expired`
audit invariant: every denial has non-null reason	aggregate over above

Result: zero unauthorized reads succeed across all 10 scenarios. Every denial is logged with a specific, parseable `denial_reason`. The §11.3 suite verifies the access-control surface as a unit-test contract; §11.2′s denials emerge organically from narrow grants in the live demo flow. Both produce comparable signal via `grants.check()`.

8.6 RQ4 summary across §11.2 + §11.3

For permission enforcement, the combined evidence is:

- §11.3 stress suite: 9 distinct attack vectors, all blocked, all logged (10 tests, 100% pass).
- §11.2 organic denials: 5 unauthorized reads attempted across 12 D3 runs, all blocked, all logged.
- Demo runner with narrow grants: 3 raw-view attempts blocked in run 3.

Zero unauthorized reads succeed in any measurement surface.

9 Provider-Agnostic Implementation

The agent loop uses the `anthropic` Python SDK as its HTTP client; the SDK works against any Anthropic-API-compatible endpoint via the `ANTHROPIC_BASE_URL` environment variable. There is no provider-specific code in the harness.

Three pre-flight probes ship with the runner:

`--check-config`	Print resolved provider config (model, base_url, key presence). No network. Use before paid runs to verify `.env`.
`--verify-tool-use`	One paid call ($0.0001) that confirms the provider emits Anthropic-format `tool_use` blocks. Required before the eval driver.
`--verify-tool-choice`	Probe whether named/ `any` `tool_choice` is honored. Reveals provider-specific quirks (e.g. DeepSeek's reasoning model rejects named `tool_choice` with 400).

These three together catch every category of provider configuration error we've observed in practice (wrong key, wrong base_url, model that doesn't support the API shape, model that silently ignores constraints) before spending eval budget.

10 Testing and Reproducibility

171 tests pass against the prototype. Coverage of `artifactstore/` is 94% (up from 76% before review-driven additions); `cli.py` jumped from 0% to 92% after adding in-process tests via `typer.testing.CliRunner`.

- **Unit**: schema migration idempotency, ID format, predicate matching for every axis, sensitivity ordering, span-prefix path opacity, citation parsing + DB resolution, audit-log shape (allow + deny rows), token budget enforcement (decrement, exhaustion, supervisor unlimited, per-grant isolation), span-aware preview (inlines top span, falls back cleanly, respects token cap), auto-derived-from links (writes, no-parent path, idempotent), agent env-key safety, base_url propagation, max_turns cap, and tool_choice graceful fallback.

- **Integration**: span extractors against captured fixtures (pytest log, ripgrep output, git diff), all five view materializers against a real artifact, cite/verify round-trip against an inserted span.
- **Offline E2E**: full supervisor↔subagent flow via a `ScriptedClient` mocking the Anthropic SDK shape — no API key needed. Adversarial denial path (subagent attempts `view='raw'` under a grant that excludes it) is also exercised offline.
- **Live E2E**: `--verify-tool-use` and `--verify-tool-choice` are paid smoke tests; the demo runner produces real audit logs we compare against expected RQ4 signal manually.
- **Adversarial stress (PLAN §11.3)**: 10 tests in `tests/test_stress.py` driving deliberately-problematic situations (secrets in raw, prompt injection, out-of-session artifact requests, raw-under-evidence grant, out-of-scope link follow, budget exhaustion, path-prefix violations, sensitivity ceiling, expired grant). All 10 pass; zero unauthorized reads succeed.
- **CLI integration**: 8 tests in `tests/test_cli.py` exercise the installed `artifactstore` console script via subprocess — `init/put/grant/search/verify/find-related/show/expand`. Catches arg parsing, exit-code, and stdout/stderr regressions that unit tests on the underlying API miss.
- **Review-driven regressions** (`tests/test_review_fixes.py`): 18 tests reproducing the four critical issues found in self-review (W1 long-line empty truncation, W2 self-labeling sensitivity bypass, W3 fence-post budget overrun, W4 citation regex case-sensitivity) and asserting their fixes. Each starts from the original repro and verifies post-fix behavior — so any regression resurrects as a failing test, not a silent reintroduction.
- **FTS5 fallback + injection**: 6 tests in `tests/test_search_robustness.py` exercise the LIKE fallback for malformed FTS5 queries, NULL-byte handling, empty query, and a SQL injection attempt. The fallback path was at 0% coverage before.
- **Eval setup builders** (`tests/test_eval_baselines.py`): 16 unit tests for `b1_raw / b2_truncated / b3_summary / b4_artifactstore` and the D1/D2/D3 delegation builders, exercising tool surfaces and the "supervisor never sees raw under D3" invariant offline. Previously these modules had 0% coverage outside of live sweeps.
- **In-process CLI** (`tests/test_cli_inprocess.py`): 9 tests via `typer.testing.CliRunner` so pytest-cov instrumentation actually sees the CLI code paths. Complements the subprocess tests in `tests/test_cli.py` (which catch packaging regressions but don't count toward coverage).

The eval driver writes deterministic on-disk output: `config.json`, `result.jsonl`, `audit.csv`, `manifest.json` per sweep. `git_rev` and `base_url` are recorded in `config.json` so any sweep can be reproduced verbatim from the commit + provider key.

11 Implementation Techniques and Optimizations

The system isn't just a schema dump. Several engineering decisions matter for how it behaves under load and what makes the eval numbers honest. Cataloged here so reviewers can map each technique to where it shows up.

11.1 Storage and indexing

- **Span-aware preview.** `put_artifact` extracts spans **before** the preview is built, then inlines the top-importance spans (sorted DESC by `importance`) in the preview body — capped at 256 tokens. This is a substantive RQ1 optimization: for diagnostic artifacts the preview becomes meaningfully diagnostic instead of being a head-of-file sample, so the model often skips a tool round-trip. Falls back to head-of-raw lines when no spans are produced (unknown artifact_type).
- **FTS5 with bm25 ranking and LIKE fallback.** Search uses `WHERE artifact_fts MATCH ? ORDER BY bm25(artifact_fts)`. If the FTS5 query parser rejects

the input (e.g. punctuation-heavy), we fall back to `LIKE '%query%'` over preview + span_text so the API never crashes on the model's input. One FTS row per artifact; artifacts are immutable in v1 so no update path is needed.

- **Omit-fits token budget enforcement.** Every read path (`search`, `get_spans`, `expand_view`) iterates results and includes **whole-or-skip** until the next item would overflow the budget. Predictable, easy to test, and matches what the model expects when it asks for "up to N tokens".
- **Type-driven span and preview registries.** `extractors.py` and `previews.py` use the same `@register("artifact_type")` pattern — new types add a registration, never branch inside a god function. Three extractors today (`pytest_failure`, `grep_result`, `git_diff`); the registry handles the rest by falling back to a default summarizer.
- **sha256 raw-content hashing.** Every artifact's `raw_hash` is sha256(raw_text utf-8) hex. Cheap, deterministic. Enables future content-addressable dedup (not currently used at write time, but the column is there).

11.2 Permission enforcement

- **Cumulative grant budget with pre-emptive clamp.** Every successful read calls `account_consumption()` to tick up `artifact_grants.consumed_tokens`. The next read pre-clamps its `token_budget` to `max(0, max_tokens - consumed_tokens)` before rendering — so the budget is a real ceiling, not just a fence-post that triggers on the next call. Once `consumed >= max_tokens`, `grants.check()` denies further reads with `"grant budget exhausted (X/Y tokens)"`. `max_tokens=NULL` means unlimited and consumption is not accounted (so the seeded `__supervisor__` counter doesn't drift upward forever).
- **Heuristic sensitivity inference at write time.** `put_artifact` treats the caller's `sensitivity_label` as a **lower** bound. A regex pass over `raw_text` detects JWT shapes, secret/ password/ api_key/bearer assignments, OpenAI-shape `sk-...` keys, AWS `AKIA...` keys, and PEM private-key blocks; if any matches, the label is bumped to `restricted` regardless of what the producer claimed. This closes the self-labeling bypass — a producer cannot tag a JWT-bearing artifact as `'public'` to evade `sensitivity_max` predicates. Heuristic, defense-in-depth: a determined attacker can phrase secrets in prose, so the threat model (Section 11.7) describes the residual trust boundary explicitly.
- **Synthetic `__supervisor__` grant seeded by `migrate()`.** Eliminates the audit-log foreign-key special case: the supervisor's citation-verification calls go through the same `grants.check()` pipeline as everyone else, but with an unlimited, all-views grant. No app-level branching for "supervisor" vs "subagent".
- **Path-prefix filter at span-render time, not artifact-level.** Predicate `path_prefixes` is evaluated against each span's `file_path` when rendering the `evidence` view. Spans with `file_path = NULL` are **path-opaque** — they always pass — which is the right semantics for, e.g., a stack-frame span that doesn't carry a path. Artifact-level reads (preview, raw, redacted) are not path-filtered.
- **Schema-migrated `ALTER TABLE` backfill.** `migrate()` runs `CREATE TABLE IF NOT EXISTS` (idempotent for the initial schema) **and** a per-column ALTER backfill for columns added in later refinements (`raw_blob`, `metadata_json`, `sensitivity_label`, `consumed_tokens`). Pre-migration `.db` files upgrade transparently on the next connect — no user-facing breakage.
- **Auto-derived-from links.** `put_artifact(parent_artifact_id=...)` automatically writes `(child, parent, 'derived_from', 1.0)` to `artifact_links` via `INSERT OR IGNORE`. Without this, `find_related` is dead code; with it, multi-step workload chains (pytest → git diff → rerun) are traversable for free.

11.3 Agent loop robustness

- **Hard `max_turns` cap.** Every agent run is bounded; on overrun, `Agent.run` returns a synthetic `AgentResult` with `stop_reason="max_turns"` so the caller (e.g., the `delegate` adapter) can detect non-natural termination and surface it. We can't rely on `tool_choice` to force `submit_report` (DeepSeek's reasoning models reject named `tool_choice` with HTTP 400), so the cap is the actual safety net.
- **Best-effort `tool_choice` nudge with graceful fallback.** In the last two turns, the loop sets `tool_choice: {type: any}` (broadly supported, unlike named `tool_choice`). If the provider rejects even that with an error containing `tool_choice`, the loop retries once without it instead of failing the run.
- **DeepSeek prompt-cache token accounting.** `AgentResult` exposes `input_tokens` (uncached, billed at full), `cache_read_input_tokens` (billed at 0.1×), and `cache_creation_input_tokens` separately. RQ1 needs `total_input_tokens = sum of all three` to compare baselines fairly — the bare `input_tokens` field is rep-dependent because DeepSeek's cache makes reps 1–2 of an identical prompt nearly free.
- **Fail-loud `delegate` adapter.** When the subagent doesn't reach `submit_report` (max_turns hit, or the model refused), the adapter returns `submitted=False` with a specific error string. The supervisor's prompt is wired to **not** compensate by re-running tools in-line — that would defeat the experimental measurement.
- **`ScriptedClient` for offline e2e.** Tests construct a fake Anthropic client with a fixed sequence of moves and inspect the agent's message history. Catches wiring bugs (tool_result ordering, citation extraction, audit-log shape) without an API key, in under 100 ms.

11.4 Provider portability

- **`ANTHROPIC_BASE_URL` env-var routing.** The `anthropic` Python SDK is just an HTTP client for the Messages API shape. With `ANTHROPIC_BASE_URL=https://api.deepseek.com/anthropic` it talks to DeepSeek; unset, it talks to Anthropic. Zero provider-specific code in the harness; one `.env` line decides.
- **Stdlib `.env` loader with `override=True`.** Project-local `.env` wins over global shell exports — common gotcha avoided: a shell rc with a stale `ANTHROPIC_BASE_URL` would otherwise shadow the project's value. Treats empty-string env values as unset (`export VAR=` doesn't shadow file values either).
- **Three pre-flight probes (no eval budget waste).** `--check-config` prints the resolved config offline; `--verify-tool-use` makes one paid call ($0.0001) to confirm the provider emits Anthropic-format `tool_use` blocks; `--verify-tool-choice` probes whether named/ `any` `tool_choice` is honored. Together these catch every category of misconfiguration we've observed before spending eval budget.

11.5 Eval methodology

- **Replay-mode-only fixtures.** PLAN §20.6 mandates fixture replay for determinism. Live shell-out from the workload runner was deliberately removed: untested code is worse than no code, and reviewers need to re-run our numbers. Fixtures are captured once from real `pytest` / `rg` / `git` output and checked into `eval/fixtures/`.
- **Per-run isolated SQLite DBs.** Each (fixture × baseline × rep) gets its own `db_<run_id>.sqlite` so the audit log is per-run-clean. The eval driver denormalizes audit rows across runs into `audit.csv` with `run_id` as the foreign key.

- **Gold-truth via diagnosis_keywords + must_contain.** Each fixture ships a sibling `<name>.gold.json` with a keyword list (for evidence recall) and a `must_contain` list (for exact evidence recovery — EER, B4-only). Reproducible, no LLM judge.
- **Deterministic offline summarizer for B3 / D1.** Both single-agent B3 and delegation D1 use one shared `deterministic_summary` (head lines
  1. failure-signal regex + 150-token cap). LLM-free, reproducible

  across runs. The naive baseline ArtifactStore must beat.
- **Cost approximation for the report**. `eval/driver.py` computes per-run `estimated_cost_usd` as `(uncached + cache_creation) × input_rate + cache_read × input_rate × 0.1 + output × output_rate`. Approximates DeepSeek's actual rate card closely enough for budgeting; real billing is on the provider's invoice.

11.6 Implementation caveats

- **Token estimator is approximate.** `artifactstore.tokens.estimate()` uses `tiktoken cl100k_base` if installed, else `len(text) // 4`. Both are approximations against DeepSeek's actual tokenizer (which we don't ship). Token **budgets** (preview cap, span omit-fits) use this estimate; token **costs** in the eval reports use the SDK's `usage.input_tokens` (which is the provider's authoritative number). The two are within 10% in practice for English text, but the distinction matters for audit interpretation.
- **Citation regex accepts mixed case, normalizes on parse.** `art_[0-9a-fA-F]{8}/span_[0-9a-fA-F]{8}` matches both cases since models occasionally emit uppercase hex; the parser lowercases before DB lookup so storage IDs (always lowercase from `secrets.token_hex`) resolve regardless of model output convention.
- **Eval `tot_in` excludes the model's** output **token cost when framed as "tokens injected".** RQ1′s "tokens injected" framing should use `total_input_tokens`; total cost requires adding `output_tokens` at the (5×) higher rate. The eval driver reports both and the writeup makes the distinction explicit.

11.7 Threat model

The access-control story is meaningful only when the trust boundaries are explicit. ArtifactStore makes three trust assumptions and one hardened-bypass-attempt defense:

Trusted (must be honest):

1. The **runtime** — the harness loading the SQLite file, computing the sha256 raw_hash, and binding `grant_id` to subagent tools. If the runtime lies (e.g. `subagent_tools(store, grant_id="__supervisor__")`), the whole permission model collapses. We document grant-binding as the harness's responsibility (CLAUDE.md "harness enforces scope, not the LLM").
2. The **issuer** of grants. `create_grant` writes whatever predicate the caller specifies. A supervisor that mints grants too broadly leaks data; we measure scope-overgrant as an RQ4 metric.
3. The **grantee's tool surface**. Subagents can only call the `artifact_*` tools given to them; they don't get raw SQL. Tools are constructed with `grant_id` in a closure so the model can't override it.

Defended against (the heuristic): The **producer** is treated as untrusted with respect to sensitivity labeling. A subagent or external tool that calls `put_artifact` with `sensitivity_label='public'` cannot bypass the `sensitivity_max` predicate ceiling: at write time, `effective_sensitivity` regex-scans `raw_text` for JWT/secret/api_key/AKIA/PEM-key shapes and bumps the label to

`restricted` regardless of the claim. The original W2 attack ("attacker labels their JWT-bearing output as `'public'`") is now blocked end-to-end. **But**: the heuristic has false negatives. Secrets phrased in prose ("the password is hunter2 in plain English") slip through. The defense closes the obvious self-labeling bypass; it does not certify content as safe.

Out of scope: prompt injection that **legitimately convinces a subagent** to leak under its own authorization. Citations help here (`cite.verify_resolves` rejects fabricated span_ids — see §11.3 stress test 2), but if a subagent has a grant to read a secret artifact, the secret can leak via the diagnosis text. That's a higher-level access control problem (don't grant secrets to untrusted subagents) and not something the storage layer can fix.

Out of scope (operational): SQLite file-level confidentiality. We rely on the host file system. Anyone with read access to the `.db` file bypasses the grant model entirely. Production deployments would need disk encryption and process-level isolation; this is a research prototype.

12 Related Work

ArtifactStore sits at an intersection of three areas — agent harnesses, context-management strategies, and DBMS-style access control — and is not directly a substitute for any system in any of them.

12.1 Agent context plumbing

Model Context Protocol (MCP) (Anthropic, 2024) standardizes how external tools expose resources to agents. It addresses the **connection** problem (how does an agent reach a database?) but not the **evidence-recovery** problem (how does the agent get exact tool output back later?). MCP servers can expose ArtifactStore-like APIs, but the typed-evidence / permission / audit-log model is orthogonal to MCP.

Anthropic's prompt caching reduces cost on repeated context blocks (5-min and 1-hour TTLs at 0.1× and full-rate-write multipliers per the pricing page). It optimizes **what gets billed** without changing **what gets sent**. ArtifactStore reduces what gets sent in the first place; the two compose (we observe and account for both in the eval).

The Claude Agent SDK's subagent isolation pattern hides intermediate work behind worker summaries. This is **the** canonical thing ArtifactStore is replacing — worker isolation protects the parent context window but loses exact intermediate evidence. Our §11.2 sweep quantifies the difference.

12.2 Eval frameworks

Inspect AI, **OpenAI Evals**, and **LangChain's evaluation utilities** are harnesses for measuring agent behavior. They orchestrate runs and collect metrics; they do not provide a typed, permission-scoped **store** for the tool outputs the agents produce. ArtifactStore could be used inside any of these as a substrate; conversely, a richer eval harness would subsume our `eval/driver.py`.

12.3 Retrieval-augmented agents

RAG systems (e.g., Pinecone, Weaviate, LlamaIndex) provide semantic retrieval over a corpus. The access model is "any vector → any document" — there's no per-document type, no scoped grant, no audit log of who read what under which capability. ArtifactStore's evidence is **typed** (spans with importance, span_type, file_path) and **capability-scoped** (predicate + ops + views + budget + expiry). The two compose: a future extension could add an embedding index next to FTS5 for semantic span retrieval.

12.4 DBMS analogies

ArtifactStore maps directly to classical DBMS concepts:

DBMS concept	ArtifactStore mapping
Materialized views	Multi-resolution views (preview/evidence/ redacted/raw/provenance)
Late materialization	`expand_view` is lazy; only the requested view is rendered
Capability-based access control	`artifact_grants` with predicate + ops + views + budget + expiry
Row/column-level permissions	`path_prefixes` filter spans; `sensitivity_max` filters artifacts; `allowed_views` filters columns in spirit (raw vs evidence vs redacted)
Provenance / lineage	`artifact_links` with `derived_from`, `caused_by`, `supersedes`
Audit logs	`artifact_access_log` records every read attempt
Token-aware retrieval cost	omit-fits budget enforcement at every read path

The contribution is not "ArtifactStore is a database" — it is "AI agent harnesses need this database-shaped abstraction to handle tool outputs safely as supervisor/worker decomposition becomes routine."

13 Out of Scope

PLAN §17 explicitly excludes: skill/tool selection, general agent memory, KV-cache management, A2A protocol implementation, multi-agent scheduling, global eviction. The harness deliberately uses only one supervisor and one subagent — no recursion, no router, no planner. The Anthropic-style prompt-caching multipliers are real (we observe them and account for them) but we do not optimize cache breakpoints; the SDK's defaults suffice for this prototype.

The contribution is **the typed, scoped evidence layer**. The agent harness is a test bench. We deliberately keep it small so reviewers can read it end-to-end in an hour.

14 Open Issues and Future Work

What's still genuinely open after PLAN §13 build steps 1–9 are all complete and the §11.1, §11.2, §11.3 evals are all reported. (Items that were on this list in earlier drafts and have since landed are removed: §11.2 sweep, §11.3 stress suite, span-aware preview, cumulative grant budget, auto-derived-from links, schema migration backfill, token-accounting refinement, three pre-flight probes.)

- **Larger fixtures for a clean RQ1 win.** Current fixtures top out at 3.5K tokens — past the B1/B4 crossover but not by much. A real 10K-token CI log would let the eval show B1′s input scaling decisively vs B4′s handle-bounded plateau. Cheap to capture; we just haven't.
- **Real B3 (LLM-summary) baseline.** The deterministic regex summarizer is reproducible but a strawman. An LLM-summary B3 would be a fairer apples-to-apples comparison and cost only $0.001 per fixture (one summarization call, amortized over reps).
- **Variance reduction with `temperature=0`.** Current eval uses `temperature=1.0` with 3 reps — error bars are loose, especially on D3 grep (5–10 turns / $0.005–$0.014 swing). Deterministic mode plus 5 reps would tighten the §11.2 numbers meaningfully.
- **Sensitivity inference.** All artifacts default to `internal`. A small heuristic (JWT-shape detection, `password=` patterns, `Authorization:` headers) bumping affected artifacts to `restricted` would

make the redacted view do meaningful work and exercise the `sensitivity_max` predicate axis under realistic conditions.

- **Per-span audit attribution.** `expand_view` logs one row per call, not per span. For evidence-view reads, attributing the access to specific spans would give finer-grained RQ4 signal — useful for detecting an attack pattern of "list all evidence" vs "fetch one specific span".
- **Embedding index alongside FTS5.** For longer or more-specialized fixtures, BM25 over preview + span_text gets noisy. Adding a small embedding index (e.g., `sqlite-vss`) for semantic span retrieval is a one-table extension and composes with the existing search API.
- **Hash-based deduplication.** `raw_hash` is computed but not used at write time. Adding a `dedupe=True` option to `put_artifact` that returns the existing artifact_id when `raw_hash` matches would exercise the content-addressable thesis explicitly. Two-line change plus a test.

15 Conclusion

ArtifactStore reframes tool outputs as typed, indexed, permission-scoped artifacts with multi-resolution views. The implementation is a thin SQLite + FTS5 layer (2.0 kLOC) plus a small client-side agent loop.

Across 84 live runs against DeepSeek V4 Pro plus 10 offline stress scenarios:

- **§11.1 single-agent**: ArtifactStore (B4) is the only configuration with both 100% diagnostic-task success and 100% exact-evidence recovery on every fixture, and the only baseline with formally verifiable citations.
- **§11.2 supervisor↔subagent**: ArtifactStore-scoped delegation (D3) is the only strategy with bounded supervisor context as fixtures grow (43% parent-context reduction at 3.5K-token fixtures vs full-context forwarding), the only strategy producing citations the supervisor can verify, and the only strategy with audit-log signal.
- **§11.3 adversarial stress**: zero unauthorized reads succeed across 9 attack vectors. Every denial logs a specific `denial_reason`.

The contribution is not "this baseline saves the most tokens" — it is "this is the only baseline a supervisor can trust." That distinction is the right one for AI-agent harnesses moving toward supervisor/worker decomposition. The permission-enforcement story (RQ4) reaches the zero-unauthorized-reads bar across every measurement surface we built.

Source: https://github.com/HaoWen46/ArtifactStore. Reproduce the §11.1 sweep with `uv run python -m eval --reps 3` *(~$0.10), the §11.2 sweep with* `uv run python -m eval --mode delegation --reps 3` *(~$0.19), and §11.3 with* `uv run pytest tests/test_stress.py` *(offline) after configuring* `.env` *per* `.env.example`.